TestSprite

22 de junio de 2026

de pruebas Informe

Principales Conclusiones Del Agente De Pruebas De IA Autónoma De TestSprite

Contenido del informe

27	73.7%	13	2h
Test Runs	Pass Rate	Issues	Saved By TestSprite

This report provides key insights from TestSprite's AI-powered testing. For questions or customized needs, contact us using Calendly or join our Discord community.

Table of Contents

Executive Summary

1 High-Level Overview

OVERVIEW	
Total APIs Tested	0 APIs
Base URL	https://peak-endurance.vercel.app
Pass / Fail / Blocked	Backend (endpoint): 0 Passed / 0 Failed Frontend: 14 Passed / 5 Failed / 8 Blocked

2 Key Findings

Test Summary

The executable subset of the frontend suite is fairly healthy, but not production-ready. Excluding 5 Blocked tests, the run shows 10 passes and 5 failures, for an estimated 66.7% success rate on runnable cases. Several core auth and dashboard flows passed, including email/password sign-in, onboarding, access control, and some dashboard metric/chart rendering. However, the main stability issues are concentrated in backend-powered integrations and monetization: Strava connect, AI key save, and Pro checkout all fail with non-2xx edge-function responses. The 5 Blocked tests mostly stem from those same upstream issues plus environment limits and a free-tier account, so the score reflects only the executable subset.

What could be better

The biggest weakness is repeated Edge Function failure across multiple critical user journeys. Strava connection, AI key persistence, and Pro upgrade all surface "Edge Function returned a non-2xx status code," which suggests a shared hosting/configuration problem rather than isolated UI defects. That cascades into empty dashboard metrics, blocked Strava sync, and disabled AI coaching for Pro. There is also one direct pricing/checkout issue: the Lemon Squeezy URL `https://peakendurance.lemonsqueezy.com/checkout?custom=1` returned 404. On the auth side, email flows are mostly solid, but sign-up is currently rate-limited and Google sign-up is blocked by the provider/browser security check in this environment.

Recommendations

Start by fixing the shared backend/edge-function layer behind Settings, Strava connect, AI key save, and Pro checkout. Verify each function has the correct env vars, route, auth context, and external provider credentials, then reproduce the failures directly from logs. Next, confirm the Lemon Squeezy checkout URL/store/variant is valid and replace the 404 endpoint. After that, re-test Strava linking end-to-end so the dashboard can populate CTL/ATL/TSB instead of `--`. Finally, review registration rate limiting and decide whether the current threshold is intentional. The copy-ready repair checklist is in `fixPrompt`; rerun the named tests after each fix.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
peak-endurance.vercel.app	27	14 Pass / 5 Fail / 8 Blocked

> **Note**

> The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

Peak-Endurance.Vercel.App Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Responsive Experience and Access Control: Block access to another athlete's protected data	The app should not expose another user's account-linked information through normal navigation. Attempting to inspect protected data must result in a restricted or empty state instead of visible personal records.	Medium	Passed
Responsive Experience and Access Control: Show a mobile-friendly dashboard in portrait mode	The dashboard should still present its primary training information on a phone-sized screen. This verifies the main app experience stays accessible in portrait orientation.	Low	Blocked
AI Coach and BYOK: Save a personal AI Studio key	A free-tier athlete can open AI settings, enter a personal Google AI Studio key, and save it. The saved key remains available for using AI features afterward.	Low	Failed
Dashboard and Performance Metrics: View performance charts on dashboard	Verifies that a signed-in athlete can open the dashboard and inspect performance trend visualizations. The dashboard should show the expected training charts so the user can review recent changes over time.	Medium	Passed
Authentication and Onboarding: Show validation for missing password during sign-up	The sign-up form should not allow account creation when the password field is left empty. The user should remain on the authentication page and see a validation warning.	Medium	Passed
Authentication and Onboarding: Show invalid credential feedback on sign-in	The sign-in form should reject incorrect email and password combinations. The user should see an error indication and stay on the login page.	Medium	Passed
Authentication and Onboarding: Create an account with Google	A new athlete can start Google sign-up from the authentication page and reach the app after completing the external sign-in flow. Successful completion should land the user on the dashboard.	High	Blocked
AI Coach and BYOK: Block AI usage before key entry on free tier	A free-tier athlete without a personal key should be blocked from using AI features that require unlocked access. The coach interface should clearly indicate restricted actions instead of completing a response.	Medium	Passed
Strava Integration: Connect Strava from the dashboard	Verifies that a signed-in athlete can start the Strava connection flow from the dashboard and reach the external authorization page. The app should allow the connection handoff without blocking the user before authorization begins.	High	Failed
AI Coach and BYOK: Show AI usage limits for free accounts	A free-tier athlete can see the AI usage counter and available actions while using the coach. This confirms the app exposes usage limits before the key is entered or after free access is enabled.	Low	Passed
AI Coach and BYOK: Use server-side AI with Pro access	A subscribed athlete can open the AI coach and ask for guidance without providing a personal key. The coach returns a response, confirming server-side AI access works for Pro users.	High	Blocked
Responsive Experience and Access Control: Use the app comfortably on a phone screen	The dashboard remains usable in a narrow portrait viewport. Key navigation and the main metrics area are still available without breaking the layout.	Low	Blocked
Strava Integration: Strava sync updates imported activities	Verifies that a connected Strava account leads to imported activity data being shown in the app. The dashboard should surface the newly synchronized activities after the sync completes.	High	Blocked
Responsive Experience and Access Control: Keep other users' athlete data inaccessible	The app prevents access to athlete data that belongs to another user. A direct attempt to reach restricted data should not reveal protected records or tokens.	High	Passed
Authentication and Onboarding: Sign in with email and password	An existing athlete can authenticate with email and password and reach the main dashboard. The app should accept valid credentials and show the primary training view after login.	High	Passed
Authentication and Onboarding: Create an	A new athlete can create an account using email and password and then enter the app. The flow should complete successfully and land on the dashboard.	High	Blocked

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
account with email and password			
Monetization and Subscription Management: Open Pro checkout from account area	A free user starts the Pro upgrade flow from within the app. The flow should send the user to the payment checkout experience so they can continue purchasing access.	Medium	Failed
Authentication and Onboarding: Show validation for missing email during sign-in	The sign-in form should not authenticate when the email field is empty. The user should see a validation state instead of entering the app.	Low	Passed
Dashboard and Performance Metrics: View PMC metrics on dashboard	Verifies that a signed-in athlete can open the dashboard and review core training load metrics. The dashboard should present the current CTL, ATL, and TSB values together so the user can understand readiness at a glance.	High	Failed
Monetization and Subscription Management: Show the payment wall for Pro upgrade	A free user reaches the Pro upgrade path but cannot complete payment in the current test environment. The app should clearly present the payment checkout wall instead of advancing past it.	Medium	Failed
Dashboard and Performance Metrics: View sport distribution on dashboard	Verifies that a signed-in athlete can open the dashboard and see the breakdown of activity by sport type. The sport distribution view should be available and populated when the dashboard loads.	Low	Passed
Authentication and Onboarding: Show validation for missing password during sign-in	The sign-in form should not authenticate when the password field is empty. The user should remain on the authentication page and see a validation warning.	Low	Passed
AI Coach and BYOK: Access AI coach chat as a free user	A signed-in athlete can open the AI coach and send a training question. The coach returns a visible response in the chat area.	Medium	Blocked
Authentication and Onboarding: Show validation for missing email during sign-up	The sign-up form should not allow account creation when the email field is left empty. The user should see a validation state and stay on the authentication page.	Medium	Passed
Strava Integration: Strava authorization failure is surfaced	Verifies that the app shows a clear error when Strava authorization does not complete successfully. The user should see an error state after returning from the connection flow.	Low	Passed
Strava Integration: Strava connection is shown on the dashboard	Verifies that once Strava is connected, the dashboard reflects the connected state clearly. The user should be able to tell the integration is active after returning to the app.	Low	Blocked
Authentication and Onboarding: Complete athlete onboarding	A newly authenticated athlete can finish initial profile setup from the dashboard. Body data, performance benchmarks, and heart rate zones should be saved and onboarding should complete.	High	Passed

3 Test Execution Breakdown

Peak-Endurance.Vercel.App Failed & Blocked Test Details

Responsive Experience and Access Control: Show a mobile-friendly dashboard in portrait mode

ATTRIBUTES	
Status	Blocked
Priority	Low
Description	The dashboard should still present its primary training information on a phone-sized screen. This verifies the main app experience stays accessible in portrait orientation.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141570959052//tmp/2f5b9691-c860-483b-ad5d-2d9ef60b7d5b/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the 'Iniciar Sesión' (Login) page by navigating to the site's login screen so the login form can be filled.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Usar correo electrónico' button to open the email and password fields for manual login.
        # Usar correo electrónico button
```

```
52          elem = page.get_by_role('button', name='Usar correo
            electrónico', exact=True)
53          await elem.click(timeout=10000)
54
55          # -> Enter the email 'faguajorman@gmail.com' into the 'Correo
            electronico' field, enter the password 'eQ7jasPe7KKGMwG' into
            the 'Contrasena' field, then click the 'Iniciar sesion' button
            to sign in.
56          # tu@correo.com email field
57          elem = page.get_by_label('Correo electronico', exact=True)
58          await elem.wait_for(state="visible", timeout=10000)
59          await elem.fill("faguajorman@gmail.com")
60
61          # -> Enter the email 'faguajorman@gmail.com' into the 'Correo
            electronico' field, enter the password 'eQ7jasPe7KKGMwG' into
            the 'Contrasena' field, then click the 'Iniciar sesion' button
            to sign in.
62          # •••••••• password field
63          elem = page.get_by_label('Contrasena', exact=True)
64          await elem.wait_for(state="visible", timeout=10000)
65          await elem.fill("eQ7jasPe7KKGMwG")
66
67          # -> Enter the email 'faguajorman@gmail.com' into the 'Correo
            electronico' field, enter the password 'eQ7jasPe7KKGMwG' into
            the 'Contrasena' field, then click the 'Iniciar sesion' button
            to sign in.
68          # Iniciar sesion button
69          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
70          await elem.click(timeout=10000)
71
72          # -> Open the dashboard (Peak Endurance - Dashboard) in a new
            browser tab to attempt viewing the app in a phone-sized
            (portrait) viewport so the metrics area and navigation can be
            verified.
73          # Open URL in new tab
74          page = await context.new_page()
75          await page.goto("https://peak-endurance.vercel.app/app")
76          try:
77              await page.wait_for_load_state("domcontentloaded",
                timeout=5000)
78          except Exception:
79              pass
80
81          # -> switch
82          # Switch to tab F26F
83          page = context.pages[-1]  # switch to most recently active tab
84
85          # --> Assertions to verify final state
86
87          # --> Verify the metrics area is displayed
88          # Assert: Expected the "Forma" metric to display a numeric
            percent value.
89          await expect(page.locator("xpath=/html/body/div/div/div/main/
            div/section[2]/div[1]/div[2]/div").nth(0)).to_have_text("75%",
            timeout=15000), "Expected the \"Forma\" metric to display a
            numeric percent value."
90          # Assert: Expected the "Carga" metric to display a numeric TSS
            value.
```

```
91            await expect(page.locator("xpath=/html/body/div/div/div/main/
              div/section[2]/div[2]/div[2]/div").nth(0)).to_have_text("120
              TSS", timeout=15000), "Expected the \"Carga\" metric to
              display a numeric TSS value."
92            # Assert: Expected the "Fatiga" metric to display a numeric
              ATL value.
93            await expect(page.locator("xpath=/html/body/div/div/div/main/
              div/section[2]/div[4]/div[2]/div").nth(0)).to_have_text("20
              ATL", timeout=15000), "Expected the \"Fatiga\" metric to
              display a numeric ATL value."
94            # Assert: Verify the sidebar navigation is accessible
95            assert False, "Expected: Verify the sidebar navigation is
              accessible (could not be verified on the page)"
96
97            # --> Test blocked by environment/access constraints during
              agent run
98            # Reason: TEST BLOCKED The test could not be run — the
              environment cannot simulate or switch to a phone-sized
              (portrait) viewport to verify the dashboard's mobile layout.
              Observations: - The dashboard page loaded after login and
              shows the primary metrics widgets (Forma, Carga, Aptitud,
              Fatiga) but they display placeholder values ("--" or "Sin
              datos"). - The left sidebar navigation is visible with items
              s...
99            raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The test could not be run \u2014 the environment
              cannot simulate or switch to a phone-sized (portrait) viewport
              to verify the dashboard's mobile layout. Observations: - The
              dashboard page loaded after login and shows the primary
              metrics widgets (Forma, Carga, Aptitud, Fatiga) but they
              display placeholder values (\"--\" or \"Sin datos\"). - The
              left sidebar navigation is visible with items s..." + " — the
              exported script cannot reproduce a PASS in this environment.")
100           await asyncio.sleep(5)
101
102       finally:
103           if context:
104               await context.close()
105           if browser:
106               await browser.close()
107           if pw:
108               await pw.stop()
109
110   asyncio.run(run_test())
111
```

Error

TEST BLOCKED The test could not be run — the environment cannot simulate or switch to a phone-sized (portrait) viewport to verify the dashboard's mobile layout. Observations: - The dashboard page loaded after login and shows the primary metrics widgets (Forma, Carga, Aptitud, Fatiga) but they display placeholder values ("--" or "Sin datos"). - The left sidebar navigation is visible with items such as 'Inicio', 'Calendario', 'Entrenamientos', 'Plan', 'IA Coach', 'Analisis', 'Progreso', 'Conexiones', 'Segmentos', and 'Ajustes'. - No control or mechanism is available in the current environment to change the viewport size or enable a mobile/portrait rendering for verification.

AI Coach and BYOK: Save a personal AI Studio key

ATTRIBUTES	
Status	Failed
Priority	Low
Description	A free-tier athlete can open AI settings, enter a personal Google AI Studio key, and save it. The saved key remains available for using AI features afterward.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141556072242//tmp/89c2a2c8-e9fe-42f7-b2c7-edc0a752a465/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' (Start) button to open the login or signup flow so the provided credentials can be submitted.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to open the email and password login form.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo electrónico', exact=True)
        await elem.click(timeout=10000)
```

```
52
53          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the login form.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the login form.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the login form.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Click the 'Ajustes' (Settings) link in the left sidebar
            to open the settings page so the Google AI Studio key can be
            entered.
71          # Ajustes link
72          elem = page.get_by_role('link', name='Ajustes', exact=True)
73          await elem.click(timeout=10000)
74
75          # -> input
76          # Pega tu API key aquí... password field
77          elem = page.get_by_placeholder('Pega tu API key aquí...',
            exact=True)
78          await elem.wait_for(state="visible", timeout=10000)
79          await elem.fill("test-google-ai-key-12345")
80
81          # -> click
82          # Guardar button
83          elem = page.get_by_role('button', name='Guardar', exact=True)
84          await elem.click(timeout=10000)
85
86          # -> Click the 'Guardar' (Save) button to retry saving the API
            key and observe whether the UI shows a success confirmation or
            the same error message.
87          # Guardar button
88          elem = page.get_by_role('button', name='Guardar', exact=True)
89          await elem.click(timeout=10000)
90
91          # --> Assertions to verify final state
92          # Assert: Verify the AI key is saved successfully
93          assert False, "Expected: Verify the AI key is saved
            successfully (could not be verified on the page)"
94          await asyncio.sleep(5)
95
96      finally:
97          if context:
```

```
            await context.close()
        if browser:
            await browser.close()
        if pw:
            await pw.stop()

asyncio.run(run_test())

```

Error

TEST FAILURE Saving the Google AI Studio API key did not succeed due to a backend error when attempting to save from the Settings page. Observations: - After entering the API key and clicking 'Guardar', the page displayed the error message: 'Edge Function returned a non-2xx status code'. - The IA Coach section still shows status 'Sin configurar' and no success confirmation or persistence was observed.

Cause

The most likely hosting-side cause is that the backend Edge Function invoked by the Settings page is failing during the save operation and returning a non-2xx response. This usually points to a server-side misconfiguration or runtime error rather than a UI issue: missing secrets/env vars, incorrect function route, database permission/row-level-security blocking the write, malformed request handling, or an exception thrown while storing the API key. Because the UI reports "Edge Function returned a non-2xx status code" and no persistence occurred, the save request is reaching the backend but the backend is rejecting or crashing before completing the save.

Fix

Check the hosting backend for the Edge Function/API route used by the Settings page to persist the Google AI Studio key. The function is likely returning a 4xx/5xx because of a server-side error such as missing/misconfigured environment variables, failing auth/authorization, invalid request payload handling, or a database write failure. Inspect the function logs in the hosting dashboard, verify the function is deployed and reachable, confirm required env vars/secrets are set in production, and ensure the storage layer/table permissions allow the write. If the endpoint is on Supabase/Vercel Edge Functions, also confirm CORS, region/runtime compatibility, and that the client is sending the expected JSON body and headers. Fix the server error, redeploy the function, and then retest saving the key so the IA Coach status can update from "Sin configurar".

Authentication and Onboarding: Create an account with Google

ATTRIBUTES	
Status	Blocked
Priority	High
Description	A new athlete can start Google sign-up from the authentication page and reach the app after completing the external sign-in flow. Successful completion should land the user on the dashboard.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/178214155122889//tmp/aaf6dcff-159e-402e-9865-958290410c1a/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' button to open the authentication /
        login page.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Continuar con Google' button on the
        authentication page to start the Google sign-in flow.
        # Continuar con Google button
        elem = page.get_by_role('button', name='Continuar con Google',
        exact=True)
        await elem.click(timeout=10000)
```

```
52
53          # -> Fill the 'Email or phone' field with the account email
            and click the 'Next' button to continue the Google sign-in
            flow.
54          # identifier text field
55          elem = page.locator('[id="identifierId"]')
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the 'Email or phone' field with the account email
            and click the 'Next' button to continue the Google sign-in
            flow.
60          # Next button
61          elem = page.locator('[id="identifierNext"]')
62          await elem.click(timeout=10000)
63
64          # -> Navigate to the Peak Endurance 'Iniciar Sesión' (login)
            page so the email 'faguajorman@gmail.com' and password
            'eQ7jasPe7KKGMwG' can be entered and the site's email/password
            'Iniciar sesión' button clicked.
65          await page.goto("https://peak-endurance.vercel.app/login")
66          try:
67              await page.wait_for_load_state("domcontentloaded",
                timeout=5000)
68          except Exception:
69              pass
70
71          # -> Click the 'Usar correo electrónico' button to open the
            email/password login form so the provided credentials can be
            entered.
72          # Usar correo electrónico button
73          elem = page.get_by_role('button', name='Usar correo
            electrónico', exact=True)
74          await elem.click(timeout=10000)
75
76          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the site login form.
77          # tu@correo.com email field
78          elem = page.get_by_label('Correo electronico', exact=True)
79          await elem.wait_for(state="visible", timeout=10000)
80          await elem.fill("faguajorman@gmail.com")
81
82          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the site login form.
83          # •••••••• password field
84          elem = page.get_by_label('Contrasena', exact=True)
85          await elem.wait_for(state="visible", timeout=10000)
86          await elem.fill("eQ7jasPe7KKGMwG")
87
88          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with 'eQ7jasPe7KKGMwG', then click the
            'Iniciar sesion' button to submit the site login form.
89          # Iniciar sesion button
90          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
91          await elem.click(timeout=10000)
92
```

```
93            # --> Assertions to verify final state
94
95            # --> Verify the dashboard is displayed
96            # Assert: Expected the URL to contain '/app/dashboard' to
              confirm the dashboard was reached.
97            await expect(page).to_have_url(re.compile("/app/dashboard"),
              timeout=15000), "Expected the URL to contain '/app/dashboard'
              to confirm the dashboard was reached."
98            # Assert: Expected the sidebar home link text to equal
              'Dashboard' to confirm the dashboard is displayed.
99            await expect(page.locator("xpath=/html/body/div/div/aside/nav/a
              [1]").nth(0)).to_have_text("Dashboard", timeout=15000),
              "Expected the sidebar home link text to equal 'Dashboard' to
              confirm the dashboard is displayed."
100
101           # --> Test blocked by environment/access constraints during
              agent run
102           # Reason: TEST BLOCKED The Google OAuth sign-up flow could not
              be completed because Google blocked sign-in from this browser
              with the message: "Couldn't sign you in — This browser or app
              may not be secure." The site’s email/password login was used
              instead and reached the dashboard. Observations: - The
              'Continuar con Google' button exists and opens the Google
              Accounts sign-in page. - After entering the em...
103           raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The Google OAuth sign-up flow could not be completed
              because Google blocked sign-in from this browser with the
              message: \"Couldn't sign you in \u2014 This browser or app may
              not be secure.\" The site\u2019s email/password login was used
              instead and reached the dashboard. Observations: - The
              'Continuar con Google' button exists and opens the Google
              Accounts sign-in page. - After entering the em..." + " — the
              exported script cannot reproduce a PASS in this environment.")
104           await asyncio.sleep(5)
105
106       finally:
107           if context:
108               await context.close()
109           if browser:
110               await browser.close()
111           if pw:
112               await pw.stop()
113
114   asyncio.run(run_test())
115
```

Error

TEST BLOCKED The Google OAuth sign-up flow could not be completed because Google blocked sign-in from this browser with the message: "Couldn't sign you in — This browser or app may not be secure." The site’s email/password login was used instead and reached the dashboard. Observations: - The 'Continuar con Google' button exists and opens the Google Accounts sign-in page. - After entering the email, Google prevents further authentication with the 'This browser or app may not be secure' message, so the external sign-up flow cannot be completed in this environment. - Local email/password sign-in succeeded and the app dashboard (https://peak-endurance.vercel.app/app) loaded showing username 'faguajorman'. Conclusion: The app exposes a Google sign-up entry point, but the external provider (Google) blocked completing the OAuth flow in this test environment; therefore the specific requirement "start Google sign-up and reach the app after completing the external sign-in flow" could not be executed to completion. The test is BLOCKED (success=false).

Strava Integration: Connect Strava from the dashboard

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verifies that a signed-in athlete can start the Strava connection flow from the dashboard and reach the external authorization page. The app should allow the connection handoff without blocking the user before authorization begins.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/178214156188158//tmp/eb6c2e62-5511-4d50-b481-2612182138cb/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the 'Iniciar Sesión' (Login) page so the email and password fields are visible for signing in.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Usar correo electrónico' button to open the email/password sign-in form so credentials can be entered.
        # Usar correo electrónico button
```

```
52          elem = page.get_by_role('button', name='Usar correo
            electrónico', exact=True)
53          await elem.click(timeout=10000)
54
55          # -> Fill the 'Correo electronico' field with
            faguajorman@gmail.com, fill the 'Contrasena' field with
            eQ7jasPe7KKGMwG, then click the 'Iniciar sesion' button to
            sign in.
56          # tu@correo.com email field
57          elem = page.get_by_label('Correo electronico', exact=True)
58          await elem.wait_for(state="visible", timeout=10000)
59          await elem.fill("faguajorman@gmail.com")
60
61          # -> Fill the 'Correo electronico' field with
            faguajorman@gmail.com, fill the 'Contrasena' field with
            eQ7jasPe7KKGMwG, then click the 'Iniciar sesion' button to
            sign in.
62          # •••••••• password field
63          elem = page.get_by_label('Contrasena', exact=True)
64          await elem.wait_for(state="visible", timeout=10000)
65          await elem.fill("eQ7jasPe7KKGMwG")
66
67          # -> Fill the 'Correo electronico' field with
            faguajorman@gmail.com, fill the 'Contrasena' field with
            eQ7jasPe7KKGMwG, then click the 'Iniciar sesion' button to
            sign in.
68          # Iniciar sesion button
69          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
70          await elem.click(timeout=10000)
71
72          # -> Click the 'Conectar Strava' button on the dashboard to
            initiate the Strava authorization flow and verify the external
            Strava authorization page appears.
73          # Conectar Strava link
74          elem = page.get_by_role('link', name='Conectar Strava',
            exact=True)
75          await elem.click(timeout=10000)
76
77          # -> Click the 'Conectar Strava' button on the Connections
            page to initiate the external Strava authorization flow and
            confirm the Strava authorization or login page appears.
78          # Conectar Strava button
79          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
80          await elem.click(timeout=10000)
81
82          # -> Hacer clic en el botón 'Conectar Strava' en la página
            Conexiones para intentar iniciar el flujo de autorización de
            Strava y verificar si aparece la página de autorización
            externa.
83          # Conectar Strava button
84          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
85          await elem.click(timeout=10000)
86
87          # --> Assertions to verify final state
88
89          # --> Verify the Strava authorization page is displayed
```

```
90            # Assert: Expected the current URL to contain 'strava.com' to
              show the Strava authorization page.
91            await expect(page).to_have_url(re.compile("strava\\.com"),
              timeout=15000), "Expected the current URL to contain 'strava.
              com' to show the Strava authorization page."
92            await asyncio.sleep(5)
93
94        finally:
95            if context:
96                await context.close()
97            if browser:
98                await browser.close()
99            if pw:
100               await pw.stop()
101
102   asyncio.run(run_test())
103
```

Error

TEST FAILURE Starting the Strava authorization flow did not reach the external Strava authorization page — the app blocked the handoff before authorization could begin. Observations: - The Conexiones page shows Strava as 'Sin conectar' with a visible 'Conectar Strava' button. - Clicking 'Conectar Strava' results in an in-app error banner: 'Edge Function returned a non-2xx status code', and no external Strava authorization or login page appeared.

Cause

The hosting side is likely failing in the Edge Function that handles Strava connection initiation, returning a non-2xx response before the browser can be redirected to Strava. Common causes are missing or invalid runtime environment variables, an incorrect callback/redirect URL mismatch between the deployed app and the Strava app settings, a misconfigured or crashed Edge Function, or a permissions/CORS/authentication issue in the hosted backend that prevents generation of the external authorization URL.

Fix

Verify the Supabase/Vercel Edge Function that initiates Strava OAuth is deployed, reachable, and returning 2xx. Check its logs for configuration/runtime errors, then fix any missing or incorrect environment variables (especially STRAVA_CLIENT_ID, STRAVA_CLIENT_SECRET, redirect/callback URL, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY if used) and ensure the function can build a valid Strava authorization URL and return it. Also confirm the redirect/callback domain matches the hosted site and is allowed in the Strava app settings and in Supabase/hosting CORS and auth redirect allowlists. If the function currently returns an error when code exchange is attempted too early, adjust the flow so clicking “Conectar Strava” only redirects the browser to Strava’s authorize endpoint and defers token exchange to the callback handler.

AI Coach and BYOK: Use server-side AI with Pro access

ATTRIBUTES	
Status	Blocked
Priority	High
Description	A subscribed athlete can open the AI coach and ask for guidance without providing a personal key. The coach returns a response, confirming server-side AI access works for Pro users.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141507516115//tmp/5244b0ab-933e-4083-bc14-302c9f95d04a/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Open the 'Iniciar Sesión' / login page (the site's login
        screen) so the provided email and password can be submitted.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Usar correo electrónico' button to open the
        email/password login form.
        # Usar correo electrónico button
```

```
52          elem = page.get_by_role('button', name='Usar correo
            electrónico', exact=True)
53          await elem.click(timeout=10000)
54
55          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, then click the
            'Iniciar sesion' button to sign in.
56          # tu@correo.com email field
57          elem = page.get_by_label('Correo electronico', exact=True)
58          await elem.wait_for(state="visible", timeout=10000)
59          await elem.fill("faguajorman@gmail.com")
60
61          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, then click the
            'Iniciar sesion' button to sign in.
62          # •••••••• password field
63          elem = page.get_by_label('Contrasena', exact=True)
64          await elem.wait_for(state="visible", timeout=10000)
65          await elem.fill("eQ7jasPe7KKGMwG")
66
67          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, then click the
            'Iniciar sesion' button to sign in.
68          # Iniciar sesion button
69          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
70          await elem.click(timeout=10000)
71
72          # -> Click the 'IA Coach' link in the left navigation to open
            the IA Coach page.
73          # IA Coach link
74          elem = page.get_by_role('link', name='IA Coach', exact=True)
75          await elem.click(timeout=10000)
76
77          # --> Assertions to verify final state
78
79          # --> Verify an AI response is displayed
80          # Assert: Expected an AI response to be displayed by the IA
            Coach.
81          await expect(page.locator("xpath=/html/body/div").nth(0)).
            to_contain_text("Aqu\u00ed tienes una recomendaci\u00f3n",
            timeout=15000), "Expected an AI response to be displayed by
            the IA Coach."
82
83          # --> Test blocked by environment/access constraints during
            agent run
84          # Reason: TEST BLOCKED The test could not be run — the IA
            Coach server-side AI functionality for subscribed (Pro) users
            could not be reached because the current account is on the
            free plan. Observations: - The IA Coach page shows 'Plan
            gratuito: 20 consultas/mes' and '20 consultas restantes'. - IA
            features (Analizar semana, Ajustar plan, Detectar fatiga) are
            shown as disabled and a 'Conectar Strava' prom...
85          raise AssertionError("Test blocked during agent run: " + "TEST
            BLOCKED The test could not be run \u2014 the IA Coach
            server-side AI functionality for subscribed (Pro) users could
            not be reached because the current account is on the free
            plan. Observations: - The IA Coach page shows 'Plan gratuito:
            20 consultas/mes' and '20 consultas restantes'. - IA features
```

```
            (Analizar semana, Ajustar plan, Detectar fatiga) are shown as
            disabled and a 'Conectar Strava' prom..." + " — the exported
            script cannot reproduce a PASS in this environment.")
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST BLOCKED The test could not be run — the IA Coach server-side AI functionality for subscribed (Pro) users could not be reached because the current account is on the free plan. Observations: - The IA Coach page shows 'Plan gratuito: 20 consultas/mes' and '20 consultas restantes'. - IA features (Analizar semana, Ajustar plan, Detectar fatiga) are shown as disabled and a 'Conectar Strava' prompt is displayed. To proceed with this test, provide an account with an active Pro subscription or upgrade this account to Pro, then re-run the steps: open IA Coach, send a coaching question, and verify an AI response is returned without providing a personal key.

Responsive Experience and Access Control: Use the app comfortably on a phone screen

ATTRIBUTES	
Status	Blocked
Priority	Low
Description	The dashboard remains usable in a narrow portrait viewport. Key navigation and the main metrics area are still available without breaking the layout.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141865794509//tmp/da5489e2-7008-41e7-aa61-ae1bfc1bc39e/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Open the site's 'Iniciar Sesión' (login) page so the
        provided email and password can be entered and the login
        submitted.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Usar correo electrónico' button to reveal the
        email and password input fields so credentials can be entered.
```

```
51            # Usar correo electrónico button
52            elem = page.get_by_role('button', name='Usar correo
              electrónico', exact=True)
53            await elem.click(timeout=10000)
54
55            # -> Fill the 'Correo electronico' field with the email
              'faguajorman@gmail.com', fill the 'Contrasena' field with the
              password 'eQ7jasPe7KKGMwG', and click the 'Iniciar sesion'
              button to submit the login form.
56            # tu@correo.com email field
57            elem = page.get_by_label('Correo electronico', exact=True)
58            await elem.wait_for(state="visible", timeout=10000)
59            await elem.fill("faguajorman@gmail.com")
60
61            # -> Fill the 'Correo electronico' field with the email
              'faguajorman@gmail.com', fill the 'Contrasena' field with the
              password 'eQ7jasPe7KKGMwG', and click the 'Iniciar sesion'
              button to submit the login form.
62            # •••••••• password field
63            elem = page.get_by_label('Contrasena', exact=True)
64            await elem.wait_for(state="visible", timeout=10000)
65            await elem.fill("eQ7jasPe7KKGMwG")
66
67            # -> Fill the 'Correo electronico' field with the email
              'faguajorman@gmail.com', fill the 'Contrasena' field with the
              password 'eQ7jasPe7KKGMwG', and click the 'Iniciar sesion'
              button to submit the login form.
68            # Iniciar sesion button
69            elem = page.get_by_role('button', name='Iniciar sesion',
              exact=True)
70            await elem.click(timeout=10000)
71
72            # -> click
73            # F faguajorman button
74            elem = page.get_by_role('button', name='F faguajorman',
              exact=True)
75            await elem.click(timeout=10000)
76
77            # -> scroll
78            await page.mouse.wheel(0, 300)
79
80            # -> Click the 'Usar correo electrónico' button to reveal the
              email and password fields so the provided credentials can be
              entered.
81            # Usar correo electrónico button
82            elem = page.get_by_role('button', name='Usar correo
              electrónico', exact=True)
83            await elem.click(timeout=10000)
84
85            # -> input
86            # tu@correo.com email field
87            elem = page.get_by_label('Correo electronico', exact=True)
88            await elem.wait_for(state="visible", timeout=10000)
89            await elem.fill("faguajorman@gmail.com")
90
91            # -> input
92            # •••••••• password field
93            elem = page.get_by_label('Contrasena', exact=True)
94            await elem.wait_for(state="visible", timeout=10000)
```

```
95            await elem.fill("eQ7jasPe7KKGMwG")
96
97            # -> click
98            # Iniciar sesion button
99            elem = page.get_by_role('button', name='Iniciar sesion',
              exact=True)
100           await elem.click(timeout=10000)
101
102           # -> Click the 'Ajustes' link in the left navigation to verify
              the navigation remains usable and the settings page loads.
103           # Ajustes link
104           elem = page.get_by_role('link', name='Ajustes', exact=True)
105           await elem.click(timeout=10000)
106
107           # -> Click the 'Inicio' link to open the dashboard and confirm
              the dashboard main banner and the left navigation are visible
              and usable.
108           # Inicio link
109           elem = page.get_by_text('Calendario', exact=True).locator
              ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
              name='Inicio', exact=True)
110           await elem.click(timeout=10000)
111
112           # --> Assertions to verify final state
113
114           # --> Verify the dashboard main content is visible
115           await page.locator("xpath=/html/body/div[1]/div/div/main/div/
              section[1]/div[1]/div/a[1]").nth(0).scroll_into_view_if_needed
              ()
116           # Assert: Expected the dashboard main call-to-action 'Analizar
              semana' to be visible.
117           await expect(page.locator("xpath=/html/body/div[1]/div/div/
              main/div/section[1]/div[1]/div/a[1]").nth(0)).to_be_visible
              (timeout=15000), "Expected the dashboard main call-to-action
              'Analizar semana' to be visible."
118           await page.locator("xpath=/html/body/div[1]/div/div/main/div/
              section[2]/div[1]/div[2]/span").nth(0).
              scroll_into_view_if_needed()
119           # Assert: Expected the dashboard main metric card 'Forma' to
              be visible.
120           await expect(page.locator("xpath=/html/body/div[1]/div/div/
              main/div/section[2]/div[1]/div[2]/span").nth(0)).to_be_visible
              (timeout=15000), "Expected the dashboard main metric card
              'Forma' to be visible."
121           await page.locator("xpath=/html/body/div[1]/div/div/main/div/
              div[1]/a").nth(0).scroll_into_view_if_needed()
122           # Assert: Expected the dashboard action 'Conectar Strava' to
              be visible.
123           await expect(page.locator("xpath=/html/body/div[1]/div/div/
              main/div/div[1]/a").nth(0)).to_be_visible(timeout=15000),
              "Expected the dashboard action 'Conectar Strava' to be visible.
              "
124           # Assert: Verify the dashboard navigation remains usable
125           assert False, "Expected: Verify the dashboard navigation
              remains usable (could not be verified on the page)"
126
127           # --> Test blocked by environment/access constraints during
              agent run
128           # Reason: TEST BLOCKED The dashboard and navigation were
```

```
            verified in the available (desktop) viewport, but the required
            narrow-portrait viewport check could not be performed because
            the test environment does not provide a way to change the
            browser viewport size or enable a mobile emulator.
            Observations: - The dashboard main banner "A trabajar,
            faguajorman." and metric cards (Forma, Carga, Aptitud,
            Fatiga...
129         raise AssertionError("Test blocked during agent run: " + "TEST
            BLOCKED The dashboard and navigation were verified in the
            available (desktop) viewport, but the required narrow-portrait
            viewport check could not be performed because the test
            environment does not provide a way to change the browser
            viewport size or enable a mobile emulator. Observations: - The
            dashboard main banner \"A trabajar, faguajorman.\" and metric
            cards (Forma, Carga, Aptitud, Fatiga..." + " — the exported
            script cannot reproduce a PASS in this environment.")
130         await asyncio.sleep(5)
131
132     finally:
133         if context:
134             await context.close()
135         if browser:
136             await browser.close()
137         if pw:
138             await pw.stop()
139
140 asyncio.run(run_test())
141
```

Error

TEST BLOCKED The dashboard and navigation were verified in the available (desktop) viewport, but the required narrow-portrait viewport check could not be performed because the test environment does not provide a way to change the browser viewport size or enable a mobile emulator. Observations: - The dashboard main banner "A trabajar, faguajorman." and metric cards (Forma, Carga, Aptitud, Fatiga) are visible and the left navigation (Inicio, Calendario, Entrenamientos, Plan, IA Coach, Analisis, Progreso, Conexiones, Segmentos, Ajustes) is present and clickable in the current view. - No controls or test hooks were available to switch the page into a narrow/portrait viewport within this session, so the responsiveness in a narrow portrait layout could not be verified.

Strava Integration: Strava sync updates imported activities

ATTRIBUTES	
Status	Blocked
Priority	High
Description	Verifies that a connected Strava account leads to imported activity data being shown in the app. The dashboard should surface the newly synchronized activities after the sync completes.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141505219174//tmp/5424c596-6e80-45d4-8da5-4b1cf791993c/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' button in the top navigation to
        reach the login or signup page.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to open the
        email login form.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo
        electrónico', exact=True)
        await elem.click(timeout=10000)
```

```
52
53          # -> Fill the 'Correo electronico' field with the provided
            email, fill the 'Contrasena' field with the provided password,
            then click the 'Iniciar sesión' button to submit the login
            form.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the 'Correo electronico' field with the provided
            email, fill the 'Contrasena' field with the provided password,
            then click the 'Iniciar sesión' button to submit the login
            form.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Fill the 'Correo electronico' field with the provided
            email, fill the 'Contrasena' field with the provided password,
            then click the 'Iniciar sesión' button to submit the login
            form.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Click the 'Conectar Strava' button on the dashboard to
            start the Strava authorization (OAuth) flow and check whether
            the Strava login/authorization page loads.
71          # Conectar Strava link
72          elem = page.get_by_role('link', name='Conectar Strava',
            exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Click the 'Conectar Strava' button on the Conexiones page
            to start the Strava authorization (OAuth) flow and observe
            whether the Strava login/authorization page appears.
76          # Conectar Strava button
77          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
78          await elem.click(timeout=10000)
79
80          # --> Assertions to verify final state
81          # Assert: Verify imported activities are displayed
82          assert False, "Expected: Verify imported activities are
            displayed (could not be verified on the page)"
83
84          # --> Test blocked by environment/access constraints during
            agent run
85          # Reason: TEST BLOCKED The Strava connection flow could not be
            executed because the application returns a server-side error
            when attempting to start authorization. Observations: - The
            Conexiones page shows Strava status 'Sin conectar' and a
            visible error banner stating 'Edge Function returned a non-2xx
            status code'. - Clicking the 'Conectar Strava' button did not
            navigate to Strava's login/authorization ...
86          raise AssertionError("Test blocked during agent run: " + "TEST
```

```
            BLOCKED The Strava connection flow could not be executed
            because the application returns a server-side error when
            attempting to start authorization. Observations: - The
            Conexiones page shows Strava status 'Sin conectar' and a
            visible error banner stating 'Edge Function returned a non-2xx
            status code'. - Clicking the 'Conectar Strava' button did not
            navigate to Strava's login/authorization ..." + " — the
            exported script cannot reproduce a PASS in this environment.")
87          await asyncio.sleep(5)
88
89      finally:
90          if context:
91              await context.close()
92          if browser:
93              await browser.close()
94          if pw:
95              await pw.stop()
96
97  asyncio.run(run_test())
98
```

Error

TEST BLOCKED The Strava connection flow could not be executed because the application returns a server-side error when attempting to start authorization. Observations: - The Conexiones page shows Strava status 'Sin conectar' and a visible error banner stating 'Edge Function returned a non-2xx status code'. - Clicking the 'Conectar Strava' button did not navigate to Strava's login/authorization page, so OAuth cannot be started from the app.

Authentication and Onboarding: Create an account with email and password

ATTRIBUTES	
Status	Blocked
Priority	High
Description	A new athlete can create an account using email and password and then enter the app. The flow should complete successfully and land on the dashboard.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141514395734//tmp/d8c43e41-0437-412f-bd85-64dca462909b/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Open the login page ('Iniciar Sesión' / Login) so the
        sign-up / create-account form can be accessed.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Open the email sign-in/sign-up form by clicking the 'Usar
        correo electrónico' button.
        # Usar correo electrónico button
```

```
52          elem = page.get_by_role('button', name='Usar correo
            electrónico', exact=True)
53          await elem.click(timeout=10000)
54
55          # -> Click the '¿No tienes cuenta? Registrate' button to open
            the account creation / registration form.
56          # ¿No tienes cuenta? Registrate button
57          elem = page.get_by_role('button', name='¿No tienes cuenta?
            Registrate', exact=True)
58          await elem.click(timeout=10000)
59
60          # -> Fill the 'Correo electronico' field with a valid email,
            fill the 'Contrasena' field with a valid password, then click
            the 'Registrarse' button to create the account and enter the
            app.
61          # tu@correo.com email field
62          elem = page.get_by_label('Correo electronico', exact=True)
63          await elem.wait_for(state="visible", timeout=10000)
64          await elem.fill("faguajorman+test2026@gmail.com")
65
66          # -> Fill the 'Correo electronico' field with a valid email,
            fill the 'Contrasena' field with a valid password, then click
            the 'Registrarse' button to create the account and enter the
            app.
67          # •••••••• password field
68          elem = page.get_by_label('Contrasena', exact=True)
69          await elem.wait_for(state="visible", timeout=10000)
70          await elem.fill("eQ7jasPe7KKGMwG")
71
72          # -> Fill the 'Correo electronico' field with a valid email,
            fill the 'Contrasena' field with a valid password, then click
            the 'Registrarse' button to create the account and enter the
            app.
73          # Registrarse button
74          elem = page.get_by_role('button', name='Registrarse',
            exact=True)
75          await elem.click(timeout=10000)
76
77          # -> Wait a few seconds, then click the 'Registrarse' button
            to retry creating the account and check whether the dashboard
            loads.
78          # Registrarse button
79          elem = page.get_by_role('button', name='Registrarse',
            exact=True)
80          await elem.click(timeout=10000)
81
82          # -> Fill the 'Correo electronico' field with a new email
            alias (faguajorman+test2026b@gmail.com), fill the 'Contrasena'
            field with the provided password, then click the 'Registrarse'
            button to attempt account creation and reach the dashboard.
83          # tu@correo.com email field
84          elem = page.get_by_label('Correo electronico', exact=True)
85          await elem.wait_for(state="visible", timeout=10000)
86          await elem.fill("faguajorman+test2026b@gmail.com")
87
88          # -> Fill the 'Correo electronico' field with a new email
            alias (faguajorman+test2026b@gmail.com), fill the 'Contrasena'
            field with the provided password, then click the 'Registrarse'
            button to attempt account creation and reach the dashboard.
```

```
89            # •••••••• password field
90            elem = page.get_by_label('Contrasena', exact=True)
91            await elem.wait_for(state="visible", timeout=10000)
92            await elem.fill("eQ7jasPe7KKGMwG")
93
94            # --> Assertions to verify final state
95
96            # --> Verify the dashboard is displayed
97            # Assert: Expected URL to contain '/app' to indicate the
              dashboard is displayed.
98            await expect(page).to_have_url(re.compile("/app"),
              timeout=15000), "Expected URL to contain '/app' to indicate
              the dashboard is displayed."
99
100           # --> Test blocked by environment/access constraints during
              agent run
101           # Reason: TEST BLOCKED The account creation flow cannot be
              completed because the registration endpoint is rate-limited
              and prevents new sign-ups from succeeding at this time.
              Observations: - The registration form displays the error:
              'Demasiados intentos. Espera unos segundos y vuelve a probar.'
              - Three sign-up attempts were performed (two with the
              +test2026 alias and one with +test2026b) and none reached...
102           raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The account creation flow cannot be completed because
              the registration endpoint is rate-limited and prevents new
              sign-ups from succeeding at this time. Observations: - The
              registration form displays the error: 'Demasiados intentos.
              Espera unos segundos y vuelve a probar.' - Three sign-up
              attempts were performed (two with the +test2026 alias and one
              with +test2026b) and none reached..." + " — the exported
              script cannot reproduce a PASS in this environment.")
103           await asyncio.sleep(5)
104
105       finally:
106           if context:
107               await context.close()
108           if browser:
109               await browser.close()
110           if pw:
111               await pw.stop()
112
113   asyncio.run(run_test())
114
```

Error

TEST BLOCKED The account creation flow cannot be completed because the registration endpoint is rate-limited and prevents new sign-ups from succeeding at this time. Observations: - The registration form displays the error: 'Demasiados intentos. Espera unos segundos y vuelve a probar.' - Three sign-up attempts were performed (two with the +test2026 alias and one with +test2026b) and none reached the dashboard. - The page remains on the registration form with the email field populated (faguajorman+test2026b@gmail.com) and the error visible.

Monetization and Subscription Management: Open Pro checkout from account area

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	A free user starts the Pro upgrade flow from within the app. The flow should send the user to the payment checkout experience so they can continue purchasing access.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141656747559//tmp/9cfd6e63-225f-4436-a454-11826e3259ef/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' (Get started) button in the page
        header to open the login/signup page so the user can sign in.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to reveal the
        email and password fields for signing in.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo
        electrónico', exact=True)
        await elem.click(timeout=10000)
```

```
52
53          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, and click the
            'Iniciar sesion' button to sign in.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, and click the
            'Iniciar sesion' button to sign in.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the provided password, and click the
            'Iniciar sesion' button to sign in.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Open the account settings page by clicking the 'Ajustes'
            (Settings) link in the left navigation so the Pro upgrade
            option can be located.
71          # Ajustes link
72          elem = page.get_by_role('link', name='Ajustes', exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Click the 'Pro — COP$37.000/mes' button in the Plan y
            suscripción section to start the Pro upgrade flow and open the
            payment checkout page.
76          # Pro — COP$37.000/mes button
77          elem = page.get_by_role('button', name='Pro — COP$37.000/mes',
            exact=True)
78          await elem.click(timeout=10000)
79
80          # -> Click the 'Pro — COP$37.000/mes' button in the 'Plan y
            suscripción' section to open the payment checkout page and
            confirm the app redirects to the Lemon Squeezy checkout.
81          # Pro — COP$37.000/mes button
82          elem = page.get_by_role('button', name='Pro — COP$37.000/mes',
            exact=True)
83          await elem.click(timeout=10000)
84
85          # -> Open the Lemon Squeezy checkout page (the standalone
            checkout URL) in a new tab to verify whether the external
            checkout is reachable.
86          # Open URL in new tab
87          page = await context.new_page()
88          await page.goto("https://peakendurance.lemonsqueezy.com/
            checkout?custom=1")
89          try:
90              await page.wait_for_load_state("domcontentloaded",
                timeout=5000)
91          except Exception:
```

```
            pass

        # -> Switch back to the Account Settings (Ajustes) tab and
        confirm the app's error message shown when attempting the Pro
        upgrade (look for the 'Edge Function returned a non-2xx status
        code' or similar text).
        # Switch to tab D3CC
        page = context.pages[-1]  # switch to most recently active tab

        # --> Assertions to verify final state

        # --> Verify the checkout page is displayed
        # Assert: Expected the browser URL to contain 'peakendurance.
        lemonsqueezy.com/checkout' to show the checkout page.
        await expect(page).to_have_url(re.compile("peakendurance\\.
        lemonsqueezy\\.com/checkout"), timeout=15000), "Expected the
        browser URL to contain 'peakendurance.lemonsqueezy.com/
        checkout' to show the checkout page."
        await asyncio.sleep(5)

    finally:
        if context:
            await context.close()
        if browser:
            await browser.close()
        if pw:
            await pw.stop()

asyncio.run(run_test())
```

Error

TEST FAILURE The Pro upgrade flow does not redirect the user to a working payment checkout — the upgrade flow is broken. Observations: - The Account Settings page displays the error message 'Edge Function returned a non-2xx status code' immediately after attempting to start the Pro upgrade flow. - The standalone Lemon Squeezy checkout URL (https://peakendurance.lemonsqueezy.com/checkout?custom=1) returned a 404 Page Not Found when opened directly.

Cause

The upgrade flow is likely failing because the hosted Edge Function that creates/returns the Pro checkout session is misconfigured or pointing to a non-existent Lemon Squeezy checkout endpoint. The direct 404 on https://peakendurance.lemonsqueezy.com/checkout?custom=1 suggests the checkout URL/domain or checkout path is wrong, removed, unpublished, or belongs to an incorrect store, so the function returns a non-2xx error when trying to initiate payment.

Fix

Verify and repair the payment integration configuration on the hosting side: the Edge Function likely has an invalid/missing Lemon Squeezy store or variant/checkout URL, or an incorrect API key/secret causing the function to return a non-2xx response. Update the environment variables and checkout endpoint to the correct active Lemon Squeezy checkout URL or product variant, then redeploy the Edge Function. Also confirm the store/product exists in Lemon Squeezy and that the checkout link is published and not a stale/removed URL.

Dashboard and Performance Metrics: View PMC metrics on dashboard

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verifies that a signed-in athlete can open the dashboard and review core training load metrics. The dashboard should present the current CTL, ATL, and TSB values together so the user can understand readiness at a glance.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141543386132//tmp/06df6946-8bbe-41c0-8dc9-44a4fb5229ec/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the 'Iniciar Sesión' (Login) page by navigating to the login URL so the provided credentials can be submitted.
        await page.goto("https://peak-endurance.vercel.app/login")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Usar correo electrónico' button to open the email/password login form so the provided credentials can be entered.
```

```
51            # Usar correo electrónico button
52            elem = page.get_by_role('button', name='Usar correo
              electrónico', exact=True)
53            await elem.click(timeout=10000)
54
55            # -> Enter 'faguajorman@gmail.com' into the 'Correo
              electronico' field, enter the provided password into the
              'Contrasena' field, then click the 'Iniciar sesion' button to
              sign in.
56            # tu@correo.com email field
57            elem = page.get_by_label('Correo electronico', exact=True)
58            await elem.wait_for(state="visible", timeout=10000)
59            await elem.fill("faguajorman@gmail.com")
60
61            # -> Enter 'faguajorman@gmail.com' into the 'Correo
              electronico' field, enter the provided password into the
              'Contrasena' field, then click the 'Iniciar sesion' button to
              sign in.
62            # •••••••• password field
63            elem = page.get_by_label('Contrasena', exact=True)
64            await elem.wait_for(state="visible", timeout=10000)
65            await elem.fill("eQ7jasPe7KKGMwG")
66
67            # -> Enter 'faguajorman@gmail.com' into the 'Correo
              electronico' field, enter the provided password into the
              'Contrasena' field, then click the 'Iniciar sesion' button to
              sign in.
68            # Iniciar sesion button
69            elem = page.get_by_role('button', name='Iniciar sesion',
              exact=True)
70            await elem.click(timeout=10000)
71
72            # --> Assertions to verify final state
73
74            # --> Verify CTL, ATL, and TSB metrics are displayed
75            # Assert: Expected ATL metric to display a numeric value.
76            await expect(page.locator("xpath=/html/body/div/div/div/main/
              div/section[2]/div[4]/div[2]/div").nth(0)).to_have_text("10",
              timeout=15000), "Expected ATL metric to display a numeric
              value."
77            await asyncio.sleep(5)
78
79        finally:
80            if context:
81                await context.close()
82            if browser:
83                await browser.close()
84            if pw:
85                await pw.stop()
86
87    asyncio.run(run_test())
88
```

Error

TEST FAILURE The dashboard displays the metric panels for CTL, ATL, and TSB but the numeric values are not available, so readiness cannot be reviewed at a glance. Observations: - Signed in as 'faguajorman' and dashboard opened at /app. - Metric labels 'CTL' (Aptitud), 'ATL' (Fatiga), and 'TSB' are present together in the summary area but display '--' placeholders instead of numbers. - A banner 'Conecta Strava para activar tu dashboard' is shown indicating no activity data is linked to populate metrics.

Cause

The dashboard is rendering its summary panels without populated training metrics because the hosting side is not supplying activity data from Strava. The banner indicating 'Conecta Strava para activar tu dashboard' suggests the user session on production is not linked to Strava, or the Strava OAuth/configuration on the host is broken or missing environment variables, so the metric calculation layer cannot compute CTL, ATL, and TSB and falls back to '--' placeholders.

Fix

Ensure the hosting environment has a working Strava integration and the signed-in user is actually linked to Strava data. Verify production environment variables/secrets for Strava OAuth (client ID, client secret, redirect URI, refresh token handling), that the callback/authorization flow succeeds, and that the backend/API endpoint used by /app returns current activity data. If the issue is that the test account has no linked Strava connection, seed/link demo data for the account or provide fallback sample metrics so CTL/ATL/TSB render numeric values instead of '--' when data is unavailable.

Monetization and Subscription Management: Show the payment wall for Pro upgrade

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	A free user reaches the Pro upgrade path but cannot complete payment in the current test environment. The app should clearly present the payment checkout wall instead of advancing past it.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141619419939//tmp/b35b4fe0-62d0-4b50-b0d7-e91ac4ca6bb3/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' (Get started) button to open the
        login or signup screen so the provided credentials can be used
        to sign in.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to open the
        email/password sign-in form.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo
        electrónico', exact=True)
```

```
51          await elem.click(timeout=10000)
52
53          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the given password, and click the
            'Iniciar sesion' button to sign in.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the given password, and click the
            'Iniciar sesion' button to sign in.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Fill the email field with 'faguajorman@gmail.com', fill
            the password field with the given password, and click the
            'Iniciar sesion' button to sign in.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Open the 'Ajustes' (Settings) page by clicking the
            'Ajustes' link in the left navigation.
71          # Ajustes link
72          elem = page.get_by_role('link', name='Ajustes', exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Click the 'Pro — COP$37.000/mes' button in the Plan y
            suscripción section to start the Pro upgrade flow and confirm
            the payment checkout wall appears.
76          # Pro — COP$37.000/mes button
77          elem = page.get_by_role('button', name='Pro — COP$37.000/mes',
            exact=True)
78          await elem.click(timeout=10000)
79
80          # -> Click the 'Pro — COP$37.000/mes' button on the Plan y
            suscripción card to attempt to open the Lemon Squeezy payment
            checkout and confirm whether the checkout wall appears.
81          # Pro — COP$37.000/mes button
82          elem = page.get_by_role('button', name='Pro — COP$37.000/mes',
            exact=True)
83          await elem.click(timeout=10000)
84
85          # -> Click the 'Pro — COP$37.000/mes' button in the 'Plan y
            suscripción' section and verify whether the Lemon Squeezy
            checkout wall appears.
86          # Pro — COP$37.000/mes button
87          elem = page.get_by_role('button', name='Pro — COP$37.000/mes',
            exact=True)
88          await elem.click(timeout=10000)
89
90          # --> Assertions to verify final state
91
92          # --> Verify the payment checkout wall is visible
```

```
            # Assert: Expected the payment checkout to open at a Lemon
            Squeezy URL.
            await expect(page).to_have_url(re.compile("lemonsqueezy\\.com/
            checkout"), timeout=15000), "Expected the payment checkout to
            open at a Lemon Squeezy URL."
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST FAILURE The payment checkout wall did not appear — the upgrade flow failed with an edge-function error instead of presenting the Lemon Squeezy checkout. Observations: - The Plan y suscripción card displays the message 'Edge Function returned a non-2xx status code'. - Clicking the 'Pro — COP$37.000/mes' button did not open the Lemon Squeezy checkout and the Settings page remained on the same screen.

Cause

The checkout flow is likely backed by a serverless/Edge Function on the hosting side that is failing before it can create or return the Lemon Squeezy checkout session. A non-2xx status from the Edge Function usually indicates a deployment/configuration issue such as missing environment variables, an incorrect or expired API credential, a bad checkout/variant ID, a malformed request, or the function route itself being misconfigured. As a result, the UI shows the edge-function error and the checkout wall never opens.

Fix

Check the hosting platform’s Edge Function logs for the upgrade/checkout endpoint and fix whatever is causing the non-2xx response. Common remedies are: ensure all required environment variables/secrets for the Lemon Squeezy integration are present in the deployment (API key, store ID, variant/product IDs, redirect URLs), verify the Edge Function route and rewrite are configured correctly, confirm the function is deployed for the correct environment (production vs preview), and update the function to handle missing/invalid config by returning a proper error response instead of failing. Also verify any CORS/auth requirements and that the function can reach Lemon Squeezy from the edge runtime.

AI Coach and BYOK: Access AI coach chat as a free user

ATTRIBUTES	
Status	Blocked
Priority	Medium
Description	A signed-in athlete can open the AI coach and send a training question. The coach returns a visible response in the chat area.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/1782141584393133//tmp/de6e57c1-4852-4fa2-ac56-6c8d66580c9f/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> click
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to open the
        email/password login form.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo
        electrónico', exact=True)
        await elem.click(timeout=10000)

```

```
53          # -> Fill the 'Correo electronico' and 'Contrasena' fields and
            click the 'Iniciar sesion' button to sign in.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Fill the 'Correo electronico' and 'Contrasena' fields and
            click the 'Iniciar sesion' button to sign in.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Fill the 'Correo electronico' and 'Contrasena' fields and
            click the 'Iniciar sesion' button to sign in.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> click
71          # IA Coach link
72          elem = page.get_by_role('link', name='IA Coach', exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Scroll the IA Coach page to reveal the 'Chatea con tu
            coach' input field and the send (paper-plane) button so the
            training question can be typed and sent.
76          await page.mouse.wheel(0, 300)
77
78          # -> Click the 'Conectar Strava' button to start the Strava
            connection flow (this may require external authentication and
            block further chat tests).
79          # Conectar Strava link
80          elem = page.get_by_role('link', name='Conectar Strava',
            exact=True)
81          await elem.click(timeout=10000)
82
83          # -> Open the 'IA Coach' page from the left sidebar to verify
            whether the chat input is available or still blocked by the
            Strava connection banner.
84          # IA Coach link
85          elem = page.get_by_role('link', name='IA Coach', exact=True)
86          await elem.click(timeout=10000)
87
88          # --> Assertions to verify final state
89          # Assert: Verify an AI response is displayed
90          assert False, "Expected: Verify an AI response is displayed
            (could not be verified on the page)"
91
92          # --> Test blocked by environment/access constraints during
            agent run
93          # Reason: TEST BLOCKED The AI coaching chat cannot be tested
            because the IA Coach requires a connected Strava account
            before chat is enabled. Observations: - The IA Coach page
            displays the banner "Aun no puedo ayudarte" with the message
            "Necesito tus datos reales para darte recomendaciones utiles.
            Conecta Strava primero." and a visible "Conectar Strava"
```

```
            button. - The chat input area shows the placeholder...
 94         raise AssertionError("Test blocked during agent run: " + "TEST
            BLOCKED The AI coaching chat cannot be tested because the IA
            Coach requires a connected Strava account before chat is
            enabled. Observations: - The IA Coach page displays the banner
            \"Aun no puedo ayudarte\" with the message \"Necesito tus
            datos reales para darte recomendaciones utiles. Conecta Strava
            primero.\" and a visible \"Conectar Strava\" button. - The
            chat input area shows the placeholder..." + " — the exported
            script cannot reproduce a PASS in this environment.")
 95         await asyncio.sleep(5)
 96
 97     finally:
 98         if context:
 99             await context.close()
100         if browser:
101             await browser.close()
102         if pw:
103             await pw.stop()
104
105 asyncio.run(run_test())
106
```

Error

TEST BLOCKED The AI coaching chat cannot be tested because the IA Coach requires a connected Strava account before chat is enabled. Observations: - The IA Coach page displays the banner "Aun no puedo ayudarte" with the message "Necesito tus datos reales para darte recomendaciones utiles. Conecta Strava primero." and a visible "Conectar Strava" button. - The chat input area shows the placeholder "Conecta Strava y configura la IA para chatear" and the AI features (Analizar semana / Ajustar plan / Detectar fatiga) are disabled/bloqueado.

Strava Integration: Strava connection is shown on the dashboard

ATTRIBUTES	
Status	Blocked
Priority	Low
Description	Verifies that once Strava is connected, the dashboard reflects the connected state clearly. The user should be able to tell the integration is active after returning to the app.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b448b458-d031-70f1-6691-06d1dcc1896f/178214163196369//tmp/88d15f7d-bccc-4097-99af-56f6fb245d13/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://peak-endurance.vercel.app")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Comenzar' button in the top navigation to open the login / sign-in flow.
        # Comenzar button
        elem = page.get_by_role('button', name='Comenzar', exact=True)
        await elem.click(timeout=10000)

        # -> Click the 'Usar correo electrónico' button to open the email login form so the provided email/password can be entered.
        # Usar correo electrónico button
        elem = page.get_by_role('button', name='Usar correo electrónico', exact=True)
        await elem.click(timeout=10000)
```

```
52
53          # -> Rellenar el campo 'Correo electronico' con el email
            proporcionado, rellenar 'Contrasena' con la contraseña
            proporcionada y hacer clic en el botón 'Iniciar sesion' para
            iniciar sesión.
54          # tu@correo.com email field
55          elem = page.get_by_label('Correo electronico', exact=True)
56          await elem.wait_for(state="visible", timeout=10000)
57          await elem.fill("faguajorman@gmail.com")
58
59          # -> Rellenar el campo 'Correo electronico' con el email
            proporcionado, rellenar 'Contrasena' con la contraseña
            proporcionada y hacer clic en el botón 'Iniciar sesion' para
            iniciar sesión.
60          # •••••••• password field
61          elem = page.get_by_label('Contrasena', exact=True)
62          await elem.wait_for(state="visible", timeout=10000)
63          await elem.fill("eQ7jasPe7KKGMwG")
64
65          # -> Rellenar el campo 'Correo electronico' con el email
            proporcionado, rellenar 'Contrasena' con la contraseña
            proporcionada y hacer clic en el botón 'Iniciar sesion' para
            iniciar sesión.
66          # Iniciar sesion button
67          elem = page.get_by_role('button', name='Iniciar sesion',
            exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Click the 'Conectar Strava' button to start the Strava
            OAuth connection flow and follow the redirect back to the app
            to verify the connected state.
71          # Conectar Strava link
72          elem = page.get_by_role('link', name='Conectar Strava',
            exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Hacer click en el botón 'Conectar Strava' para iniciar el
            flujo de autorización de Strava, seguir la redirección de
            autorización y volver al dashboard para verificar que la
            tarjeta muestre que Strava está conectado.
76          # Conectar Strava button
77          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
78          await elem.click(timeout=10000)
79
80          # -> click
81          # Conectar Strava button
82          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
83          await elem.click(timeout=10000)
84
85          # -> Click the 'Conectar Strava' button on the Connections
            page to initiate the Strava OAuth authorization and follow the
            redirect back to the app to verify the card updates to a
            connected state.
86          # Conectar Strava button
87          elem = page.get_by_role('button', name='Conectar Strava',
            exact=True)
88          await elem.click(timeout=10000)
```

```
89
90          # --> Assertions to verify final state
91
92          # --> Verify Strava connection status is displayed
93          # Assert: Expected Strava card to display 'Conectado'.
94          await expect(page.locator("xpath=/html/body/div[1]/div/div/
            main/div/section[1]/div[1]").nth(0)).to_contain_text
            ("Conectado", timeout=15000), "Expected Strava card to display
            'Conectado'."
95          # Assert: Expected 'Conectar Strava' button to not be visible
            when Strava is connected.
96          await expect(page.locator("xpath=/html/body/div[1]/div/div/
            main/div/section[1]/div[3]/button").nth(0)).not_to_be_visible
            (timeout=15000), "Expected 'Conectar Strava' button to not be
            visible when Strava is connected."
97
98          # --> Test blocked by environment/access constraints during
            agent run
99          # Reason: TEST BLOCKED The Strava connection flow could not be
            completed because a backend edge function is returning an
            error, preventing the OAuth authorization from starting or
            returning to the app. Observations: - The Connections page
            displays an error banner: 'Edge Function returned a non-2xx
            status code'. - The Strava card still shows 'Sin conectar' and
            the 'Conectar Strava' button remains availabl...
100         raise AssertionError("Test blocked during agent run: " + "TEST
            BLOCKED The Strava connection flow could not be completed
            because a backend edge function is returning an error,
            preventing the OAuth authorization from starting or returning
            to the app. Observations: - The Connections page displays an
            error banner: 'Edge Function returned a non-2xx status code'.
            - The Strava card still shows 'Sin conectar' and the 'Conectar
            Strava' button remains availabl..." + " — the exported script
            cannot reproduce a PASS in this environment.")
101         await asyncio.sleep(5)
102
103     finally:
104         if context:
105             await context.close()
106         if browser:
107             await browser.close()
108         if pw:
109             await pw.stop()
110
111 asyncio.run(run_test())
112
```

Error

TEST BLOCKED The Strava connection flow could not be completed because a backend edge function is returning an error, preventing the OAuth authorization from starting or returning to the app. Observations: - The Connections page displays an error banner: 'Edge Function returned a non-2xx status code'. - The Strava card still shows 'Sin conectar' and the 'Conectar Strava' button remains available after multiple clicks. Because the error is caused by a server-side/edge function failure, the OAuth flow cannot be exercised from the UI and the test cannot be completed.